

Multithreading under the lens: what is VMLens and how can it help you?

didmotoc

Who are we?



Christian Lutnik

Software Developer II at Dynatrace

- Working on OpenFeature
- Mostly Java



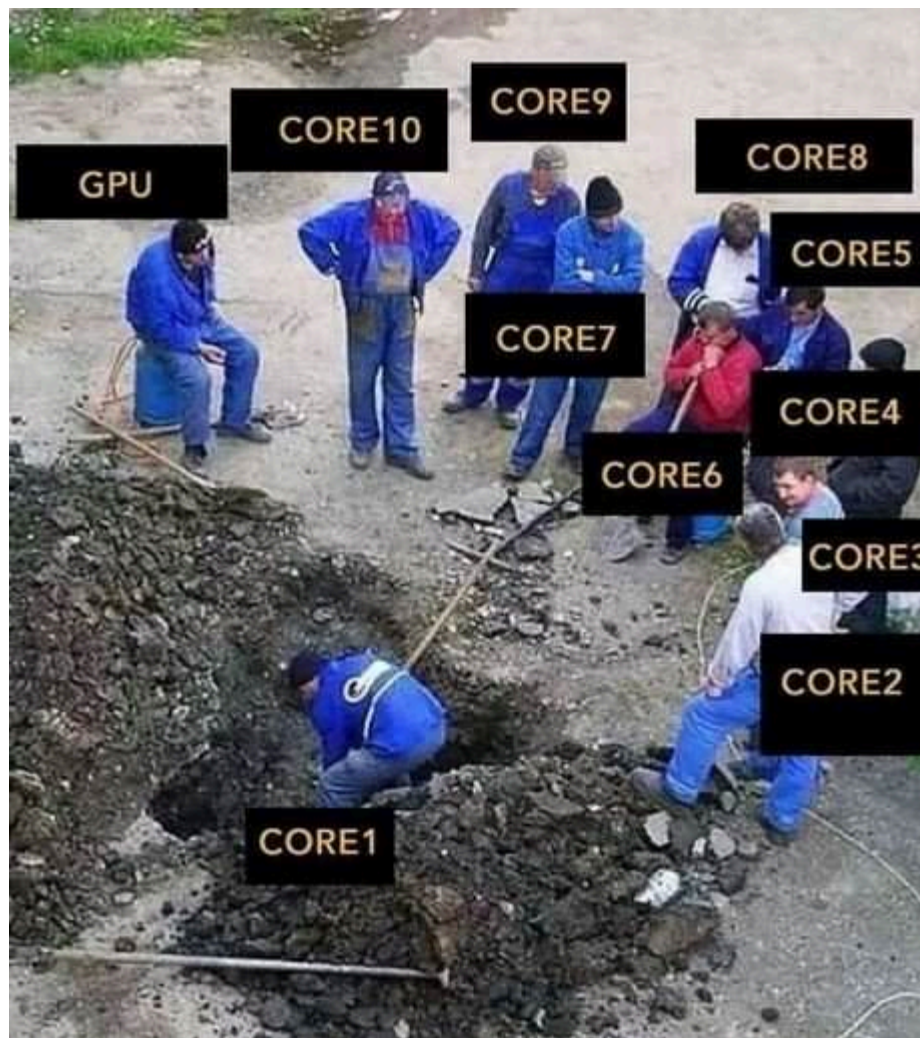
Marco Sussitz

Senior Software Developer at Dynatrace

- Java Agent Dev
- Java/C++

4 out of 5 Dev suffer from multithreading.
The one person that enjoys it





Yes, I read
"Multithreading for Dummies"

You have
exactly 10 seconds
to get the fuck
out of my house

So my daughter tells me you're
an expert at parallel programming



MULTITHREADING



VMLENS



What is VMLens?

- Concurrency testing tool for Java applications
- Detects race conditions and deadlocks at runtime
- Works with multithreaded code without manual instrumentation
- Helps developers find bugs that only appear under specific timing conditions
- Integrates with JUnit and popular Java frameworks

How does VMLens Work?

- Attaches a Java Agent
- Agent keeps track of what the code does
- Agent intercepts calls to `hasNext()`
- Creates report of test suite

Disclaimer: Java Memory Model

The following content discusses the Java Memory Model (JMM) and concurrent programming concepts.

This is a **simplified overview**. The actual JMM specification is complex and nuanced.

For production code involving concurrency:

- Consult the official Java Language Specification chapter 17

GeeCON 2018: Aleksey
Shipilëv - Java Memory Model
Unlearning Experience



Close Encounters of The Java Memory Model Kind



Safe Publication and Safe Initialization in Java



VMLens in Practice

How Does VMLens Achieve This?



What are agents?

- Provides services that allow Java programming language agents to instrument programs running on the JVM. The mechanism for instrumentation is modification of the byte-codes of methods.
- Used for profiling, debugging, and monitoring.
- `-javaagent:<jarpath>[=<options>]`

```
mvn -DargLine="-javaagent: ... /target/vmlens-agent/agent.jar" test
```



‘If you want to make
an apple pie from scratch,
you must first
create the universe’

- Carl Sagan

What happens during class loading?

- Loading
- ~~Verify~~
- Prepare
- (Optionally) Resolve

Observe when classes are loaded

```
void JNICALL  
ClassFileLoadHook(jvmtiEnv *jvmti_env,  
    JNIEnv* jni_env,  
    jclass class_being_redefined,  
    jobject loader,  
    const char* name,  
    jobject protection_domain,  
    jint class_data_len,  
    const unsigned char* class_data,  
    jint* new_class_data_len,  
    unsigned char** new_class_data)
```

Is there a better way?

- Java Agents
- `java.lang.instrument`
- `java -javaagent:agent.jar -jar helloWorld.jar`

```
1 public class SampleAgent {  
2     public static void premain(String arguments, Instrumentation instrumentationObject) {  
3         instrumentationObject.addTransformer(new OptimusPrime(), false);  
4     }  
5 }
```

```
1 public class OptimusPrime implements ClassFileTransformer {
2     @Override
3     public byte[] transform(ClassLoader loader,
4                             String className,
5                             Class<?> classBeingRedefined,
6                             ProtectionDomain protectionDomain,
7                             byte[] classFileBuffer) throws IllegalArgumentException {
8         return transformClassFile(classFileBuffer);
9     }
10
11     public byte[] transformClassFile(byte[] classfileBuffer) {
12         //TODO
13         return null;
14     }
15 }
```

- Java Meta-Agent
- a custom build JVM

Who instruments the
native instrumenters?



Recap

- TODO
- TODO
- TODO

Questions?

Andrew Strace